

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



École Nationale Supérieure d'Informatique Sidi-Bel-Abbès

Fiche TP N° 01 :

LE PRETRAITEMENT EN NLP

Dadoune Ikram

Groupe 1 - IASD -

Année Universitaire 2024-2025

Contents

1	Introduction	3
2	A. Préparation de données	3
2.1	Importation du jeu de données	3
3	B. Nettoyage d'un texte	4
3.1	1. Gérer les caractères répétitifs	4
3.2	Manipuler des homoglyphes	5
3.3	Transformer les entrées spéciales telles que les URL, e-mail et HTML à une forme canonique	6
3.4	Mettre tous les caractères en minuscule	6
3.5	Supprimer la ponctuation	7
3.6	Supprimer les mots-vide	7
3.7	Détection et traduction des langues	8
3.8	Suppression des caractères spéciaux	8
3.9	Suppression des répétitions de mots et des espaces inutiles	9
4	C. Segmentation	10
4.1	Segmentation par espaces et ponctuation	10
4.2	Segmentation basé sur des règles (avec spaCy)	10
4.2.1	Pourquoi spaCy et pas NLTK ?	10
4.3	Segmentation en sous mots (Subword Tokenization)	11
5	D. Reconnaissance d'entité nommée	12
5.1	Pour chaque phrase représentait les entités nommées (avec SpaCy).	12
5.2	Analyse des POS Tags avec SpaCy	12
6	E. Réduction des formes	13
6.1	La lemmatisation et la racinisation avec NLTK	13
7	F. Analyse des fréquences	14
7.1	Compter le nombre de phrases, pour chaque auteur, où apparaît le mot Great	14
7.2	Représentation graphique avec PyWaffle	14
7.2.1	Great graphe :	14
7.2.2	Impossible graphe:	15
7.3	Nuages de mots :	16
7.4	Nuages de mots pour les mots positifs et négatifs :	17
7.4.1	Analyse de Sentiment : Identifications de la Polarité Dominante	18
8	Analyser les données et déterminer les méthodes de prétraitement les plus adéquates pour ce dataset	20
8.1	Choix de la lemmatisation	20
8.2	Choix de la segmentation basée sur des règles	20
8.3	Nettoyage des textes	20
8.4	Conclusion	21

1 Introduction

Dans ce TP, je vais appliquer des techniques de prétraitement de texte sur le fichier `spooky.csv`. Je vais importer les données, nettoyer les textes en supprimant les caractères spéciaux, gérer les mots-vide, et effectuer des tâches comme la segmentation des phrases et la reconnaissance d'entités nommées. J'utiliserai des outils comme NLTK, SpaCy, WordCloud et Pywaffle pour analyser et visualiser les résultats. L'objectif est de choisir et justifier les méthodes de prétraitement les plus adaptées pour ce jeu de données.

2 A. Préparation de données

2.1 Importation du jeu de données

```
1 import pandas as pd
2
3 url = "https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv"
4
5 df = pd.read_csv(url)
6
7 print(df.head(10))
```

	id	text	author
0	id26305	This http://example.com pr0cesses, however, @f...	EAP
1	id17569	It #never once!every Occurred to me that the f...	HPL
2	id17564	bonjour je suis student in primaire	HPL
3	id11008	In his left hand was a gold snuff box, from wh...	EAP
4	id27763	How lovely is spring As we looked from Windsor...	MWS
5	id12958	Finding nothing else, not even gold, the Super...	HPL
6	id22965	A youth passed in solitude, my best years spen...	MWS
7	id09674	The astronomer, perhaps, at this point, took r...	EAP
8	id13515	The surcingle hung in ribands from my body.	EAP
9	id19322	I knew that you could not say to yourself 'ste...	EAP

3 B. Nettoyage d'un texte

3.1 1. Gérer les caractères répétitifs

Pour gérer les caractères répétitifs (par exemple, convertir « coooooool » en « cool »), j'ai utilisé une méthode qui combine des expressions régulières et une vérification de la validité des mots avec WordNet de NLTK. L'idée est de réduire les répétitions excessives de lettres tout en s'assurant que le mot résultant est valide en anglais.

Tout d'abord, j'ai téléchargé les ressources nécessaires de NLTK, notamment la liste de mots `words` et `wordnet`, qui contiennent un grand nombre de mots et de synonymes en anglais. Cela me permet de vérifier si un mot, après réduction des répétitions, est bien un mot valide.

```
1 from nltk.corpus import wordnet
2 import re
3 import nltk
4 nltk.download("words")
5 nltk.download("wordnet")
6
7 from nltk.corpus import words
8 english_words = set(words.words())
9
10 def reduce_repeated_chars(word):
11
12     word = re.sub(r'(\w)\1{2,}', r'\1\1', word)
13
14     if word in english_words:
15         return word
16
17
18     else:
19         word = re.sub(r'(\w)\1', r'\1', word)
20         if word in english_words:
21             return word
22
23     return word
24
25 def normalize_text(text):
26     words = text.split()
27     normalized_words = [reduce_repeated_chars(word) for word in words]
28     return " ".join(normalized_words)
29
30 sample_text = "Ooooh my Gooood , I loooove this coooooool moooooovvie! It'ssss amaaazing and
31               fantasssstic! i liiiikeee iiiitttt"
32 normalized_text = normalize_text(sample_text)
33 print(normalized_text)
```

Pourquoi j'utilise WordNet : J'utilise la liste de mots `words` de NLTK pour vérifier si le mot après réduction des répétitions est bien un mot valide. Après avoir appliqué une expression régulière pour réduire les répétitions, je vérifie si le mot résultant est un mot valide en anglais. Si ce n'est pas le cas, j'essaie de réduire encore plus les répétitions (en limitant à deux caractères seulement). Cette approche garantit que les mots résultants sont corrects et compréhensibles.

```
sample_text = "Ooooh my Gooood , I loooove this coooooool moooooovvie! It'ssss amaaazing and fantasssstic! i liiiikeee iiiitttt"
normalized_text = normalize_text(sample_text)
print(normalized_text)

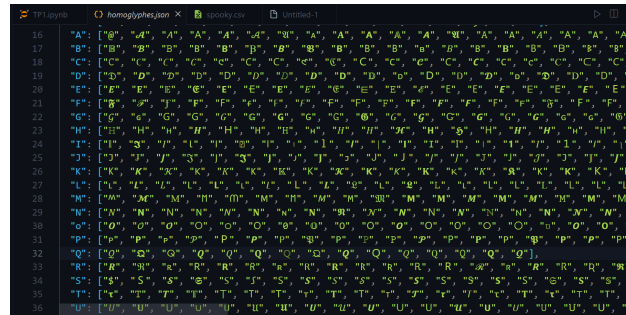
✓ 0.0s Python
Ooh my God , I love this cool movie! It's amazing and fantastic! i like it
```

3.2 Manipuler des homoglyphes

Les homoglyphes sont des caractères qui se ressemblent visuellement mais qui sont différents sur le plan informatique. Par exemple, le signe dollar *"peuttreutilislalacedelalettre"S"dans"stupide"pourcrer"tupide"*. Dans cette étape, j'ai utilisé un fichier JSON contenant une liste de caractères homoglyphes pour remplacer ces caractères par leurs équivalents corrects dans le texte.

1. J'ai créé un fichier JSON `homoglyphes.json` qui contient des paires de caractères où chaque clé est un caractère "correct" et chaque valeur est une liste de caractères "homoglyphes" pouvant être utilisés à sa place.

Exemple de contenu du fichier `homoglyphes.json` :



2. Ensuite, j'ai utilisé le fichier JSON pour remplacer ces homoglyphes dans le texte

```
1 import json
2
3 with open('homoglyphes.json', 'r', encoding='utf-8') as f:
4     char_map = json.load(f)
5
6 def replace_characters(text, char_map):
7     for key, values in char_map.items():
8         for value in values:
9             text = text.replace(value, key)
10    return text
11
12 text = "$tupide is c00l @nd I am"
13
14 cleaned_text = replace_characters(text, char_map)
15
16 print(f"Original text: {text}")
17 print(f"Cleaned text: {cleaned_text}")
```

Test du code :

```
Original text: $tupide is c00l @nd I am
Cleaned text: Stupide is cool And I am
```

3.3 Transformer les entrées spéciales telles que les URL, e-mail et HTML à une forme canonique

Lors du nettoyage de texte, il est souvent nécessaire de convertir certaines entrées spéciales, comme les URL, les adresses e-mail, les balises HTML, etc., en une forme canonique afin de simplifier le traitement et l'analyse du texte. Pour ce faire, j'ai utilisé des expressions régulières (regex) pour identifier ces éléments et les remplacer par des étiquettes standardisées telles que [URL], [EMAIL], etc.

Voici le code que j'ai utilisé pour transformer ces entrées spéciales :

```
1 def canonicalize_special_entries(text):
2     text = re.sub(r'http[s]?://\S+', '[URL]', text)
3
4     text = re.sub(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', '[EMAIL]', text)
5
6     text = re.sub(r'<.*?>', '[HTML]', text)
7
8     text = re.sub(r'\+?(?(\d{1,3})?[-.\s]?(\d{1,3})?[-.\s]?(\d{1,4})[-.\s]?(\d{1,4})', '[PHONE]',
9         text)
10
11     text = re.sub(r'(\d{1,2}[/-]\d{1,2}[/-]\d{4})|(\b[A-Za-z]+\d{1,2}, \d{4}\b|\d{4}-\d{2}-\d{2})',
12         '[DATE]', text)
13
14     text = re.sub(r'#\w+', '[HASHTAG]', text)
15
16     text = re.sub(r'@\w+', '[USER]', text)
17
18     return text
19
20 text = "Contact us at support@example.com or visit http://example.com for more info. Call
21 +1-800-555-5555, or @john_doe has #hashtag details. "
22 cleaned_text = canonicalize_special_entries(text)
23 print(cleaned_text)
```

Explication de la fonction `canonicalize_special_entries` : Cette fonction prend en entrée un texte et remplace toutes les **entrées spéciales** par des étiquettes standardisées par l'utilisation des expressions régulières. Les **entrées spéciales** traitées incluent :

- **URL** : Par exemple, `http://example.com` remplacé par [URL].
- **Adresses e-mail** : Par exemple, `support@example.com` remplacé par [EMAIL].
- **Balises HTML** : Par exemple, `<div>content</div>` remplacé par [HTML].
- **Numéros de téléphone** : Par exemple, `+1-800-555-5555` remplacé par [PHONE].
- **Dates** : Par exemple, `01/02/2022` ou `2022-02-01` remplacé par [DATE].
- **Hashtags** : Par exemple, `#hashtag` remplacé par [HASHTAG].
- **Mentions d'utilisateurs** : Par exemple, `@john_doe` remplacé par [USER].

```
Contact us at support@example.com or visit http://example.com for more info. Call +1-800-555-5555, or @john_doe has #hashtag details.
Contact us at [EMAIL] or visit [URL] for more info. Call [PHONE], or [USER] has [HASHTAG] details.
```

3.4 Mettre tous les caractères en minuscule

Cette opération permet de traiter des mots de manière identique, quel que soit le cas (ex. Hello et hello deviennent tous les deux hello).

```
1 def convert_to_lowercase(text):
2     return text.lower()
```

3.5 Supprimer la ponctuation

La méthode `text.translate(str.maketrans("", "", string.punctuation))` permet de supprimer tous les caractères de ponctuation d'un texte en utilisant une table de traduction où les caractères de ponctuation (définis dans `string.punctuation`) sont supprimés.

```
1 import string
2
3 def remove_punctuation(text):
4     return text.translate(str.maketrans('', '', string.punctuation))
5 df.dropna(subset=['text'], inplace=True)
```

De plus, on a utilisé la méthode `dropna` de Pandas pour supprimer les lignes où la colonne `text` est vide (NaN).

3.6 Supprimer les mots-vide

Les mots-vide sont des mots fréquents dans une langue mais qui n'apportent pas de valeur ajoutée pour certaines analyses. Par exemple, les mots "the", "is" en anglais.

en utilisant la liste des mots-vide fournie par la bibliothèque NLTK :

```
1 from nltk.corpus import stopwords
2
3 stop_words = set(stopwords.words('english'))
4
5 def remove_stopwords(text):
6     return ' '.join([word for word in text.split() if word not in stop_words])
```

3.7 Détection et traduction des langues

Bibliothèques utilisées :

- **langdetect** : Cette bibliothèque me permet de détecter la langue d'un texte en utilisant un modèle probabiliste basé sur des données linguistiques.
- **deep_translator.GoogleTranslator** : Cette bibliothèque me permet de traduire les textes en utilisant l'API de Google Translate, ce qui garantit une traduction rapide et efficace.

```
1 from langdetect import detect
2 from deep_translator import GoogleTranslator
3
4 df['lang'] = df['text'].apply(lambda x: detect(x) if x.strip() else 'unknown')
5
6 freq_lang = df['lang'].mode()[0]
7
8 print(df.head())
9 print("Langue la plus fr quente : ", freq_lang)
10
11 def textTraduction(text, lang):
12     detected_lang = detect(text) if text.strip() else 'unknown'
13     if detected_lang != freq_lang and detected_lang != 'unknown':
14         return GoogleTranslator(source=detected_lang, target=freq_lang).translate(text)
15     return text
16
17 df['text_traduit'] = df.apply(lambda row: textTraduction(row['text'], row['lang']), axis=1)
18
19 print(df.head())
```

```
id text author language \
0 id26305 This httpexamplecom pr0cesses however fforded ... EAP en
1 id17569 It never once Occurred to me that the fumbling... HPL en
2 id17564 bonjour je suis student in primaire HPL fr
3 id11008 In his left hand was a gold snuff box from whi... EAP en
4 id27763 How lovely is spring As we looked from Windsor... MWS en

translated_text \
0 This http://example.com pr0cesses, however, @f...
1 It #never once Occurred to me that the fumblin...
2 Hello, I am a primary school student.
3 In his left hand was a gold snuff box, from wh...
4 How lovely is spring As we looked from Windsor...
```

3.8 Suppression des caractères spéciaux

j'utilise une expression régulière qui remplace tous les caractères qui ne sont ni des lettres , ni des chiffres , ni des espaces par une chaîne vide.

```
1 import re
2
3 df['text'] = df['text'].apply(lambda text: re.sub(r'^a-zA-Z0-9\s', '', text))
```

3.9 Suppression des répétitions de mots et des espaces inutiles

Dans cette étape, je supprime les répétitions de mots consécutifs (par exemple "un un un...") et les espaces inutiles dans les textes.

```
1 def remove_repeated_words(text):
2     words = text.split()
3     cleaned_words = [words[i] for i in range(len(words)) if i == 0 or words[i] != words[i-1]]
4     return ' '.join(cleaned_words)
5
6 df['text'] = df['text'].apply(remove_repeated_words)
7 df['text'] = df['text'].str.strip()
```

Explication :

- `remove_repeated_words(text)` : Cette fonction prend un texte, le découpe en mots, puis supprime les répétitions de mots consécutifs.
- L'expression `words[i] != words[i-1]` est utilisée pour vérifier si le mot actuel est différent du précédent.
- `str.strip()` est ensuite utilisé pour supprimer les espaces inutiles aux extrémités du texte.

4 C. Segmentation

4.1 Segmentation par espaces et ponctuation

utilise une expression régulière pour diviser le texte à chaque ponctuation ou espace.

```
1 def seg_spac_ponc(text):
2     return re.split(r'[\.\!\?;\,\s]+' , text)
3
4 for i, line in df.iterrows():
5     text = line['text']
6     print(seg_spac_ponc(text))
```

```
[This, 'http://example.com', processes, 'however', 'afforded', 'me', 'no', 'means', 'bonjour', 'of', 'ascertaining', 'the', '']
[It, 'never', 'once', 'every', 'occurred', 'to', 'me', 'that', 'the', 'fumbling', 'might', 'be', 'a', 'mere', 'mistake', '']
['bonjour', 'je', 'suis', 'student', 'in', 'primaire']
[In, 'his', 'left', 'hand', 'was', 'a', 'gold', 'snuff', 'box', 'from', 'which', 'as', 'he', 'capered', 'down', 'the', 'hill', 'cu
[Now, 'lovely', 'is', 'spring', 'as', 'we', 'looked', 'from', 'Windsor', 'Terrace', 'on', 'the', 'sixteen', 'fertile', 'counties',
[Finding, 'nothing', 'else', 'not', 'even', 'gold', 'the', 'Superintendent', 'abandoned', 'his', 'attempts', 'but', 'a', 'perplex
[A, 'youth', 'passed', 'in', 'solitude', 'my', 'best', 'years', 'spent', 'under', 'your', 'gentle', 'and', 'feminine', 'fosterage
[The, 'astronomer', 'perhaps', 'at', 'this', 'point', 'took', 'refuge', 'in', 'the', 'suggestion', 'of', 'non', 'luminosity', 'and
[The, 'surcingle', 'hung', 'in', 'ribands', 'from', 'my', 'body', '']
```

4.2 Segmentation basé sur des règles (avec spaCy)

Pour segmenter un texte en phrases, j'utilise spaCy, une bibliothèque qui fournit des modèles linguistiques pré-entraînés. Contrairement à NLTK, spaCy dispose d'une fonction native pour la segmentation de phrases, `doc.sents`, qui est plus rapide et précise. Ainsi, le modèle `en_core_web_sm` est déjà pré-entraîné pour effectuer des tâches courantes de traitement du langage naturel (NLP), comme la segmentation de texte en phrases.

```
1 import spacy
2
3 nlp = spacy.load('en_core_web_sm')
4
5 def segment_with_spacy(text):
6     doc = nlp(text)
7     sentences = [sent.text for sent in doc.sents]
8     return sentences
9
10 for i, line in df.iterrows():
11     text = line['text']
12     print(segment_with_spacy(text))
```

```
[This http://example.com processes, however, afforded me no means bonjour of ascertaining the dimensions of my dungeon, as I might
[It never once every Occurred to me that the fumbling might be a mere mistake.]
['bonjour je suis student in primaire']
[In his left hand was a gold snuff box, from which, as he capered down the hill, cutting all manner of fantastic steps, he took snu
[Now lovely is spring As we looked from Windsor Terrace on the sixteen fertile counties spread beneath, speckled by happy cottages
[Finding nothing else, not even gold, the Superintendent abandoned his attempts; but a perplexed look occasionally steals over his
[A youth passed in solitude, my best years spent under your gentle and feminine fosterage, has so refined the groundwork of my chan
[The astronomer, perhaps, at this point, took refuge in the suggestion of non luminosity, and here analogy was suddenly let fall.]
[The surcingle hung in ribands from my body.]
[\"I knew that you could not say to yourself 'stereotomy' without being brought to think of atonies, and thus of the theories of Epic
[\"I confess that neither the structure of languages, nor the code of governments, nor the politics of various states possessed attra
[\"we shall find that I can feel my injuries; he shall learn to dread my revenge\" A few days after he arrived.]
```

4.2.1 Pourquoi spaCy et pas NLTK ?

- spaCy offre la fonction `doc.sents`, qui est spécifiquement conçue pour la segmentation de phrases, tandis que NLTK nécessite une approche manuelle ou des règles personnalisées.
- spaCy est plus rapide et plus précis grâce à ses modèles pré-entraînés, contrairement à NLTK qui nécessite plus de configuration pour obtenir des résultats similaires.

4.3 Segmentation en sous mots (Subword Tokenization)

Le BertTokenizer de la bibliothèque transformers est un outil utilisé pour tokeniser du texte afin de le préparer pour l'entrée dans un modèle BERT. BERT (Bidirectional Encoder Representations from Transformers) est un modèle pré-entraîné conçu pour des tâches de traitement du langage naturel telles que la classification de texte, l'analyse de sentiments, etc. Le tokenizer est essentiel pour convertir le texte brut en un format que BERT peut comprendre.

Le tokenizer fonctionne de la manière suivante :

- Il découpe le texte en unités plus petites appelées tokens.
- Il convertit ces tokens en entiers à l'aide d'un vocabulaire construit à partir d'un grand corpus de texte.
- Il gère les sous-mots et les tokens spéciaux de manière à permettre à BERT de mieux comprendre et représenter le sens du texte.

Le processus de tokenisation se déroule ainsi :

1. Charger un tokenizer BERT pré-entraîné (par exemple bert-base-uncased).
2. Tokeniser chaque morceau de texte individuellement en une séquence de tokens.
3. Chaque token est ensuite converti en son indice correspondant dans le vocabulaire du tokenizer.

Le code suivant montre comment utiliser le BertTokenizer pour tokeniser un texte contenu dans un DataFrame pandas. Le code affichera la version tokenisée de chaque entrée textuelle.

```
1 from transformers import BertTokenizer
2
3 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
4
5 for i, line in df.iterrows():
6     text = line['text']
7     print(tokenizer.tokenize(text))
```

```
['.', '.', 'example', '.', '.', 'con', 'pr', '##', '##ces', '##ses', '.', 'however', '.', 'g', 'ff', '##ord', '##ed', 'be', 'no', 'means',
'once', '.', 'even', 'occurred', 'to', 'me', 'that', 'the', 'fu', '##bling', 'might', 'be', 'a', 'more', 'mistake', '.']
['.', 'je', 'sui', '##s', 'student', 'in', 'prima', '##ire']
['hand', 'was', 'a', 'gold', 'a', '##nu', '##r', 'box', '.', 'from', 'which', '.', 'as', 'he', 'cap', '##red', 'down', 'the', 'hill',
'spring', 'as', 'we', 'looked', 'from', 'window', 'terrace', 'on', 'the', 'sistem', 'fertile', 'counties', 'spread', 'beneath',
'else', '.', 'not', 'even', 'gold', '.', 'the', 'superintendent', 'abandoned', 'his', 'attempts', '.', 'but', 'a', 'per', '##plex',
d', 'in', 'solitude', '.', 'my', 'best', 'years', 'spent', 'under', 'your', 'gentle', 'and', 'feminine', 'foster', '##age', '.', 'has',
'perhaps', '.', 'at', 'this', 'point', 'I', 'took', 'refuge', 'in', 'the', 'suggestion', 'of', 'non', 'lu', '##in', '##osity',
'is', '##is', 'hung', 'in', 'rib', '##ands', 'from', 'my', 'body', '.']
```

Ce processus permet de (Si un modèle rencontre un mot qui n'est pas dans son vocabulaire, il peut le découper en sous-mots et ainsi toujours avoir une représentation utile du mot, même s'il ne le connaît pas)

5 D. Reconnaissance d'entité nommée

5.1 Pour chaque phrase représentait les entités nommées (avec SpaCy).

```
1 import spacy
2
3 nlp = spacy.load('en_core_web_sm')
4
5 def entite_nommee(text):
6     doc = nlp(text)
7     named_entities = [(entity.text, entity.label_) for entity in doc.ents]
8     return named_entities
9
10 for i, line in df.iterrows():
11     text = line['text']
12     print(entite_nommee(text))
```

```
[[]]
[[]]
[['Windsor Terrace', 'ORG'], ('sixteen', 'CARDINAL'), ('former years', 'DATE')]
[['Superintendent', 'LOC']]
[['years', 'DATE']]
[[]]
[['Epicurus', 'PRODUCT'], ('Greek', 'NORP'), ('Orion', 'EVENT')]
[[]]
[['A few days', 'DATE']]
[[]]
[['Herbert West', 'PERSON']]
[['Wheaton Street', 'ORG']]
[[]]
```

SpaCy est plus rapide et précis que NLTK surtout lorsque nous travaillons avec des données réelles, pour la reconnaissance des entités nommées grâce à ses modèles pré-entraînés modernes et optimisés.

5.2 Analyse des POS Tags avec SpaCy

```
1 import spacy
2
3 nlp = spacy.load('en_core_web_sm')
4
5 def pos_tag(text):
6     doc = nlp(text)
7     pos_tags = [(token.text, token.pos_) for token in doc]
8     return pos_tags
9
10 for i, line in df.iterrows():
11     text = line['text']
12     print(pos_tag(text))
```

```
[('this', 'PRON'), ('http://example.com', 'N'), ('processes', 'VERB'), ('', 'PUNCT'), ('however', 'ADV'), ('', 'PUNCT'), ('offended', 'VERB'), ('it', 'PRON'), ('a', 'NOUN'), ('never', 'ADV'), ('once', 'ADV'), ('occurred', 'VERB'), ('to', 'ADP'), ('me', 'PRON'), ('th', 'PUNCT'), ('bonjour', 'NOUN'), ('', 'SPACE'), ('je', 'PRON'), ('suis', 'PRON'), ('student', 'PRON'), ('in', 'ADP'), ('primaire', 'NOUN')]
[('in', 'ADP'), ('this', 'PRON'), ('last', 'ADP'), ('hand', 'NOUN'), ('was', 'ADV'), ('a', 'DET'), ('gold', 'NOUN'), ('snuff', 'NOUN')]
[('now', 'SCONJ'), ('lovely', 'ADJ'), ('is', 'ADV'), ('spring', 'NOUN'), ('as', 'SCONJ'), ('we', 'PRON'), ('looked', 'VERB'), ('from', 'ADP'), ('finding', 'VERB'), ('nothing', 'PRON'), ('else', 'ADV'), ('', 'PUNCT'), ('not', 'PART'), ('even', 'ADV'), ('gold', 'NOUN'), ('', 'PUNCT'), ('a', 'DET'), ('youth', 'NOUN'), ('passed', 'VERB'), ('in', 'ADP'), ('solitude', 'NOUN'), ('', 'PUNCT'), ('my', 'PRON'), ('best', 'ADJ'), ('the', 'DET'), ('astronomer', 'NOUN'), ('', 'PUNCT'), ('perhaps', 'ADV'), ('', 'PUNCT'), ('at', 'ADP'), ('this', 'DET'), ('point', 'NOUN')]
[('the', 'DET'), ('surcingle', 'NOUN'), ('hung', 'VERB'), ('in', 'ADP'), ('ribands', 'NOUN'), ('from', 'ADP'), ('my', 'PRON'), ('bod', 'NOUN')]
```

L'analyse des POS tags identifie la catégorie grammaticale de chaque mot (nom, verbe, adjectif, etc.), ce qui facilite l'interprétation syntaxique et le traitement du langage naturel.

6 E. Réduction des formes

6.1 La lemmatisation et la racinisation avec NLTK

La **lemmatisation** consiste à réduire un mot à sa forme de base (lemme) en tenant compte du contexte linguistique et de la grammaire. Par exemple, le mot *better* est ramené à *good* en tant qu'adjectif.

La **racinisation** (stemming) est une approche plus rudimentaire qui tronque les mots à leur racine en supprimant les suffixes. Elle est plus rapide que la lemmatisation mais peut produire des formes incorrectes (par exemple, *running* devient *run* mais *flies* devient *fli*).

Dans ce travail, j'utilise `SnowballStemmer`, une amélioration de `PorterStemmer`, et `WordNetLemmatizer` pour la lemmatisation.

Les bibliothèques téléchargées sont :

- `punkt` : Utilisée pour la tokenisation des phrases et des mots.
- `wordnet` : Base de données lexicale utilisée pour la lemmatisation.
- `omw-1.4` : Ressources linguistiques supplémentaires pour améliorer la lemmatisation.

```
1 import nltk
2 import pandas as pd
3 from nltk.stem import WordNetLemmatizer, SnowballStemmer
4 from nltk.tokenize import word_tokenize
5
6 nltk.download('punkt')
7 nltk.download('wordnet')
8 nltk.download('omw-1.4')
9
10 lemmatizer = WordNetLemmatizer()
11 stemmer = SnowballStemmer("english")
12
13 def process_text(text):
14     words = word_tokenize(text)
15     lemmatized_text = ' '.join([lemmatizer.lemmatize(word) for word in words])
16     stemmed_text = ' '.join([stemmer.stem(word) for word in words])
17     return lemmatized_text, stemmed_text
18
19 df[['lemmatized_text', 'stemmed_text']] = df['text'].apply(lambda x: pd.Series(process_text(x)))
20
21 print(df.head())
```

```
      id      text author \
0  id26305  This http://example.com processes, however, @f...  EAP
1  id17569  It #never once!every Occurred to me that the f...  HPL
2  id17564      bonjour je suis student in primaire          HPL
3  id11008  In his left hand was a gold snuff box, from wh...  EAP
4  id27763  How lovely is spring As we looked from Windsor...  MWS

      lemmatized_text \
0  This http : //example.com processes , however ...
1  It # never once ! every Occurred to me that th...
2      bonjour je suis student in primaire
3  In his left hand wa a gold snuff box , from wh...
4  How lovely is spring As we looked from Windsor...

      stemmed_text
0  this http : //example.com process , howev , @ ...
1  it # never onc ! everi occur to me that the fu...
2      bonjour je sui student in primair
3  in his left hand was a gold snuff box , from w...
4  how love is spring as we look from windsor ter...
```

7 F. Analyse des fréquences

7.1 Compter le nombre de phrases, pour chaque auteur, où apparaît le mot Great

J'utilise la méthode `str.contains()` de pandas avec une expression régulière pour rechercher le mot *great* sous toutes ses formes (`case=False` pour insensibilité à la casse). Ensuite, je regroupe les résultats par auteur et compte les occurrences.

```
1 great_counts = df[df['text'].str.contains(r'\bgreat\b', case=False, regex=True)]\  
2                 .groupby('author').size()  
3  
4 print("Nombre de phrases contenant 'great' par auteur :")  
5 print(great_counts)
```

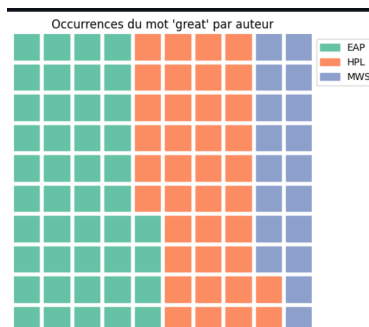
```
Nombre de phrases contenant 'great' par auteur :  
author  
EAP    212  
HPL    179  
MWS     86  
dtype: int64
```

7.2 Représentation graphique avec PyWaffle

7.2.1 Great graphe :

Pour visualiser les résultats, nous utilisons un graphique en mosaïque (*waffle chart*) avec la bibliothèque PyWaffle.

```
1 from pywaffle import Waffle  
2 import matplotlib.pyplot as plt  
3  
4 waffle_data = great_counts.to_dict()  
5  
6 plt.figure(  
7     FigureClass=Waffle,  
8     rows=5, columns=10,  
9     values=waffle_data,  
10    legend={'loc': 'upper left', 'bbox_to_anchor': (1, 1)}  
11 )  
12 plt.title("Occurrences du mot 'great' par auteur")  
13 plt.show()
```



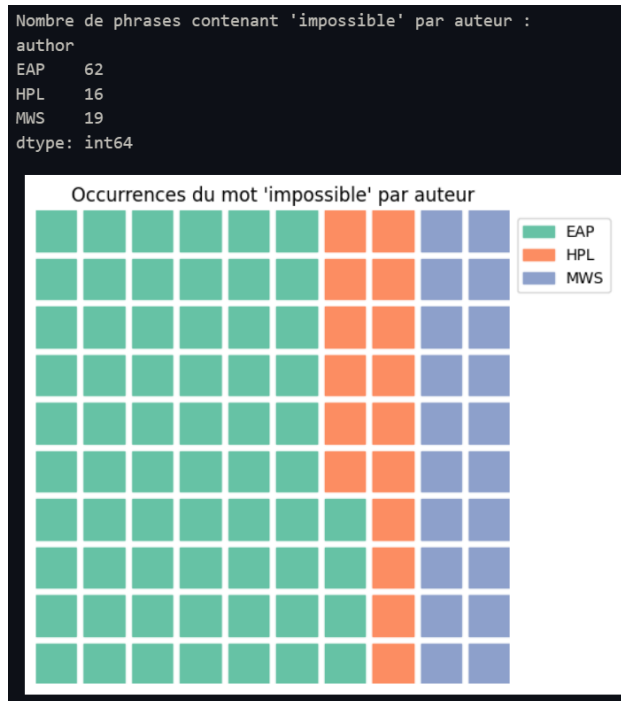
Explication :

- Les données sont converties en dictionnaire ('EAP': 30, 'HPL': 25, 'MWS': 15) avec `great_counts.to_dict()`.
- Chaque case du graphique représente une occurrence.

7.2.2 Impossible graphe:

Pour visualiser les résultats, nous utilisons un graphique en mosaïque (*waffle chart*) avec la bibliothèque PyWaffle.

```
1 from pywaffle import Waffle
2 import matplotlib.pyplot as plt
3
4 impossible_counts = df[df['text'].str.contains(r'\bimpossible\b', case=False, regex=True)].
   groupby('author').size()
5
6 print("Nombre de phrases contenant 'impossible' par auteur :")
7 print(impossible_counts)
8
9 waffle_data_impossible = impossible_counts.to_dict()
10
11 plt.figure(
12     FigureClass=Waffle,
13     rows=10, columns=10,
14     values=waffle_data_impossible,
15     legend={'loc': 'upper left', 'bbox_to_anchor': (1, 1)}
16 )
17 plt.title("Occurrences du mot 'impossible' par auteur")
18 plt.show()
```



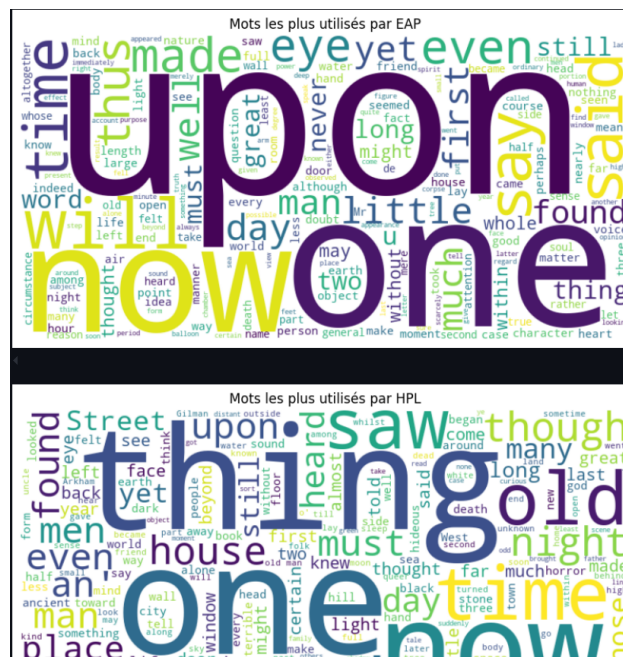
7.3 Nuages de mots :

générer trois nuages de mots représentant les termes les plus fréquents pour chaque auteur.

```

1 from wordcloud import WordCloud
2 import matplotlib.pyplot as plt
3
4 def generate_wordcloud(text, title):
5     wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)
6     plt.figure(figsize=(10, 5))
7     plt.imshow(wordcloud, interpolation='bilinear')
8     plt.axis('off')
9     plt.title(title)
10    plt.show()
11
12    authors = df['author'].unique()
13
14    for author in authors:
15        author_text = ' '.join(df[df['author'] == author]['text'])
16        generate_wordcloud(author_text, f"Mots les plus utilisés par {author}")

```



7.4 Nuages de mots pour les mots positifs et négatifs :

Pour visualiser les 100 mots les plus positifs et négatifs utilisés par chaque auteur, j'ai utilisé la fonction WordCloud et la bibliothèque NLTK : l'extraction des mots, la filtration des mots positifs et négatifs à l'aide du lexique d'opinion de NLTK, et la génération des nuages de mots.

```
1 from nltk.corpus import opinion_lexicon
2 import nltk
3 from collections import Counter
4 import re
5 from wordcloud import WordCloud
6 import matplotlib.pyplot as plt
7
8 nltk.download('opinion_lexicon')
9
10 positive_words = set(opinion_lexicon.positive())
11 negative_words = set(opinion_lexicon.negative())
12
13 authors = df['author'].unique()
14 author_texts = {author: ' '.join(df[df['author'] == author]['text']) for author in authors}
15
16 def extract_words(text):
17     return re.findall(r'\b\w+\b', text.lower())
18
19 def filter_words(words, sentiment_words):
20     return [word for word in words if word in sentiment_words]
21
22 author_positive_words = {}
23 author_negative_words = {}
24
25 for author, text in author_texts.items():
26     words = extract_words(text)
27     positive = filter_words(words, positive_words)
28     negative = filter_words(words, negative_words)
29
30     author_positive_words[author] = Counter(positive)
31     author_negative_words[author] = Counter(negative)
32
33 def generate_wordcloud(word_freq, title):
34     wordcloud = WordCloud(width=800, height=400, background_color='white').
35         generate_from_frequencies(word_freq)
36     plt.figure(figsize=(10, 5))
37     plt.imshow(wordcloud, interpolation='bilinear')
38     plt.axis('off')
39     plt.title(title)
40     plt.show()
41
42 for author in authors:
43     top_positive = dict(author_positive_words[author].most_common(100))
44     generate_wordcloud(top_positive, f"Top 100 mots positifs - {author}")
45
46     top_negative = dict(author_negative_words[author].most_common(100))
47     generate_wordcloud(top_negative, f"Top 100 mots négatifs - {author}")
```



7.4.1 Analyse de Sentiment : Identifications de la Polarité Dominante

attribuer une polarité (positive, négative ou neutre) à chaque phrase d'un texte par l'utilisation de TextBlob (polarité fréquente)

```
1 from textblob import TextBlob
2 import matplotlib.pyplot as plt
3
4 def get_polarity(text):
5     return TextBlob(text).sentiment.polarity
6
7 df['polarity'] = df['text'].apply(get_polarity)
8
9 print(df.head())
10
11 def classify_polarity(polarity):
12     if polarity > 0:
13         return 'positive'
14     elif polarity < 0:
15         return 'negative'
16     else:
17         return 'neutral'
18
19 df['sentiment'] = df['polarity'].apply(classify_polarity)
20
21 print(df.head())
22
23 sentiment_counts = df.groupby(['author', 'sentiment']).size().unstack(fill_value=0)
24
25 print("Nombre de phrases par sentiment et par auteur :")
26 print(sentiment_counts)
27
28 dominant_sentiment = sentiment_counts.idxmax(axis=1)
29
30 print("Polarité dominante pour chaque auteur :")
31 print(dominant_sentiment)
32
33 sentiment_counts.plot(kind='bar', stacked=True, figsize=(10, 6))
34 plt.title('R partition des sentiments par auteur')
35 plt.xlabel('Auteur')
36 plt.ylabel('Nombre de phrases')
37 plt.legend(title='Sentiment')
38 plt.show()
```

```
processed_text  polarity sentiment
0 url proceses however user means bonjour ascert...  0.625000  positive
1 hashtag ocured fumbling might mere mistake -0.500000  negative
2 bonjour je suis student primaire  0.000000  neutral
3 left hand gold snuff box capered hil cutting m...  0.107407  positive
4 lovely spring loked windsor terrace sixteen fer...  0.500000  positive

sentiment_counts = df.groupby(['author', 'sentiment']).size().unstack(fi

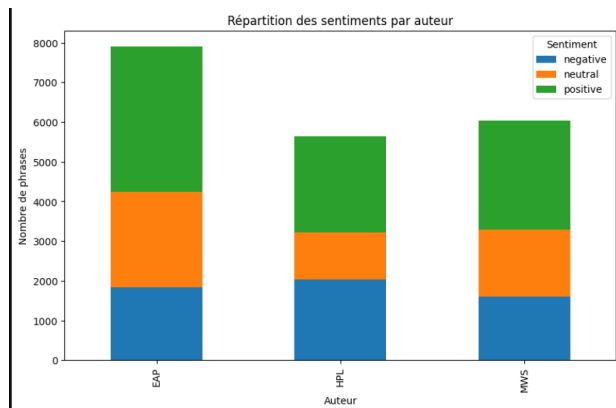
print("Nombre de phrases par sentiment et par auteur :")
print(sentiment_counts)

Nombre de phrases par sentiment et par auteur :
sentiment negative neutral positive
author
EAP          1838      2409      3653
HPL          2041      1180      2415
MWS          1598      1693      2753

dominant_sentiment = sentiment_counts.idxmax(axis=1)

print("Polarité dominante pour chaque auteur :")
print(dominant_sentiment)

Polarité dominante pour chaque auteur :
author
EAP    positive
HPL    positive
MWS    positive
dtype: object
```



8 Analyser les données et déterminer les méthodes de prétraitement les plus adéquates pour ce dataset

Dans le cadre de l'analyse des données pour ce projet, plusieurs méthodes de prétraitement peuvent être appliquées à ce dataset. En ce qui concerne la réduction des formes des mots, j'ai choisi d'utiliser la lemmatisation plutôt que la racinisation pour les raisons suivantes :

8.1 Choix de la lemmatisation

La lemmatisation consiste à réduire les mots à leur forme canonique ou "lemme". Contrairement à la racinisation, qui coupe parfois des parties de mots pour obtenir la racine (potentiellement une forme non réelle du mot), la lemmatisation permet de conserver des mots ayant du sens. Par exemple, les mots "améliorer" et "amélioration" seraient réduits à leur forme de base "améliorer". Cette approche est donc plus précise et plus respectueuse du sens des mots dans un texte, ce qui est essentiel pour les tâches de traitement du langage naturel telles que la classification ou l'analyse des sentiments.

8.2 Choix de la segmentation basée sur des règles

Pour la segmentation des phrases, j'ai opté pour un algorithme de segmentation basé sur des règles plutôt que sur une approche de tokenisation en sous-mots (Subword Tokenization). L'algorithme basé sur des règles est plus adapté pour diviser correctement le texte en unités grammaticales (par exemple, les phrases et les mots), ce qui est important pour l'analyse syntaxique et sémantique. Il utilise des règles explicites sur la ponctuation et la structure des phrases pour obtenir une segmentation précise, ce qui est essentiel pour comprendre le contexte du texte.

8.3 Nettoyage des textes

En plus des étapes de lemmatisation et de segmentation, un nettoyage approfondi des textes a été effectué pour améliorer la qualité des données d'entrée. Ce nettoyage inclut plusieurs étapes essentielles telles que la suppression des mots vides (stopwords), des URLs, et des caractères spéciaux. Voici le processus de nettoyage utilisé :

- **Canonicalisation des entrées spéciales** : Cette étape permet de convertir des entités spéciales (comme les URL et les adresses email) en une forme canonique.
- **Normalisation du texte** : Cette opération standardise les textes pour corriger les incohérences typographiques.
- **Remplacement des caractères** : Les caractères non standard sont remplacés par leur forme correcte à l'aide d'une table de correspondance.
- **Conversion en minuscules** : Cela permet d'éviter que des variantes de casse (comme "Bonjour" et "bonjour") soient traitées comme des mots différents.
- **Suppression de la ponctuation** : Tous les signes de ponctuation sont retirés pour ne garder que les mots.
- **Suppression des mots vides (stopwords)** : Les mots vides tels que "le", "la", "et", etc., qui n'apportent pas de valeur sémantique, sont supprimés.
- **Suppression des répétitions de mots** : Les répétitions inutiles (par exemple "un un un...") sont éliminées.

Le processus de nettoyage a été implémenté par la fonction suivante, appliquée à chaque texte du dataset :

```

1 def process_text(text, char_map, most_common_lang):
2     text = canonicalize_special_entries(text)
3     text = normalize_text(text)
4     text = replace_characters(text, char_map)
5     text = convert_to_lowercase(text)
6     text = remove_punctuation(text)
7     text = remove_stopwords(text)
8     text = remove_repeated_words(text)
9     return text
10
11 df['cleaned_text'] = df['text'].apply(lambda text: process_text(text, char_map, most_common_lang)
12 )
13 print(df['cleaned_text'].head(10))

```

Cette fonction permet de nettoyer les textes en enlevant les éléments non pertinents et en préparant les données pour les étapes suivantes du processus d'analyse.

8.4 Conclusion

Le choix de la lemmatisation, de la segmentation basée sur des règles et du nettoyage approfondi du texte permet d'assurer que le texte est correctement préparé pour les étapes suivantes de l'analyse. La lemmatisation offre une réduction précise des mots, la segmentation par règles fournit une découpe fiable du texte, et le nettoyage élimine les éléments inutiles qui pourraient nuire à la performance du modèle d'analyse.

df.head()

0.0%

Python

	id	text	author	lang	text traduit	language	translated text	cleaned text	canonicalized	normalized	replaced chars	lowercased	no_punctuation	no_stopwords	no_repeated_words
0	id26305	http://example.com processes, however, @f...	EAP	en	http://example.com processes, however, @f...	en	http://example.com processes, however, @f...	[url, processes, however, user, means, bonjour,...	This [URL] processes, however, [USER] me no me...	This [URL] processes, however, [USER] me no mea...	This [URL] processes, however, [USER] me no mea...	this [url] processes, however, (user) me no means bon...	url processes however user means bonjour ascet...	url processes however user means bonjour ascet...	
1	id17569	It #never once/every Occured to me that the f...	HPL	en	It #never once/every Occured to me that the f...	en	It #never once/every Occured to me that the f...	[hashtag, once/every, ocured, fumbling, might, ...	It [HASHTAG] once/every Occured to me that th...	It [HASHTAG] once/every Occured to me that the ...	It [hashtag] once/every Occured to me that the ...	it hashtag once/every ocured to me that the fum...	hashtag once/every ocured fumbling might mere m...	hashtag once/every ocured fumbling might mere m...	
2	id17564	bonjour je suis etudiant in primaire	HPL	fr	bonjour je suis etudiant in primaire	fr	Hello, I am a primary school student.	[bonjour, je, suis, etudiant, primaire]	bonjour je suis etudiant in primaire	bonjour je suis etudiant in primaire	bonjour je suis etudiant in primaire	bonjour je suis etudiant in primaire	bonjour je suis etudiant primaire	bonjour je suis etudiant primaire	
3	id11008	In his left hand was a gold snuff box, from wh...	EAP	en	In his left hand was a gold snuff box, from wh...	en	In his left hand was a gold snuff box, from wh...	[left, hand, gold, snuff, box, capered, hil, ...	In his left hand was a gold snuff box, from wh...	In his left hand was a gold snuff box, from wh...	In his left hand was a gold snuff box, from wh...	in his left hand was a gold snuff box, from wh...	left hand gold snuff box capered hil cutting m...	left hand gold snuff box capered hil cutting m...	
4	id27763	How lovely is spring As we looked from Windsor...	MWS	en	How lovely is spring As we looked from Windsor...	en	How lovely is spring As we looked from Windsor...	[lovely, spring, looked, windsor, terrace, siste...	How lovely is spring As we looked from Windsor...	How lovely is spring As we looked from Windsor ...	How lovely is spring As we looked from Windsor ...	how lovely is spring as we looked from windsor ...	lovely spring looked windsor terrace sixteen fer...	lovely spring looked windsor terrace saten sixteen fer...	